Name: Sk Arshad Basha

Reg no: 192312336

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real- world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

CODE

```
# Dataset
data = [
    ["Sunny", "Hot",  "High",   "Weak",   "No"],
    ["Sunny", "Hot",  "High",   "Strong", "No"],
    ["Cloudy", "Hot", "High",   "Weak",   "Yes"],
    ["Rainy", "Mild", "High",   "Weak",   "Yes"],
    ["Rainy", "Cool", "Normal", "Weak",   "Yes"],
    ["Rainy", "Cool", "Normal", "Strong", "No"],
    ["Cloudy", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High",   "Weak",   "No"],
    ["Sunny", "Cool", "Normal", "Weak",   "Yes"],
    ["Rainy", "Mild", "Normal", "Weak",   "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Cloudy", "Mild", "High",   "Strong", "Yes"],
    ["Cloudy", "Hot",  "Normal", "Weak",   "Yes"],
    ["Rainy", "Mild", "High",   "Strong", "No"]
]
train = data[:10]
def naive_bayes(train, test, feature_indexes, smoothing):
    classes = list(set(row[-1] for row in train))
```

```
    predictions = []
    for sample in test:
        probabilities = {}
            if row[-1] == cls
        ]
        # P(feature | class)
        for index in feature_indexes:
            count = sum(


                )
                    total + possible_values
                )
            else:
                if count == 0:
    return accuracy, precision, recall
# Experiment 1
# All Features WITHOUT Laplace Smoothing
features = [0, 1, 2, 3]
prediction1 = naive_bayes(
    train,
    test,
    features,
    False
)
accuracy, precision, recall = evaluate(
    actual,
    prediction1
```

```
)
print("EXPERIMENT 1")
print("All Features - Without Laplace Smoothing")
print("Predictions:", prediction1)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall:", round(recall * 100, 2), "%")
# Experiment 2
# All Features WITH Laplace Smoothing
prediction2 = naive_bayes(
    train,
    test,
    features,
    True
)
accuracy, precision, recall = evaluate(
    actual,
    prediction2
)
print("\nEXPERIMENT 2")
print("All Features - With Laplace Smoothing")
print("Predictions:", prediction2)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall:", round(recall * 100, 2), "%")
all:", round(recall * 100, 2), "%")
```

OUTPUT

```
EXPERIMENT 1
All Features - Without Laplace Smoothing
Predictions: ['No', 'Yes', 'Yes', 'No']
Accuracy: 75.0 %
Precision: 100.0 %
Recall: 66.67 %

EXPERIMENT 2
All Features - With Laplace Smoothing
Predictions: ['No', 'Yes', 'Yes', 'No']
Accuracy: 75.0 %
Precision: 100.0 %
Recall: 66.67 %
```

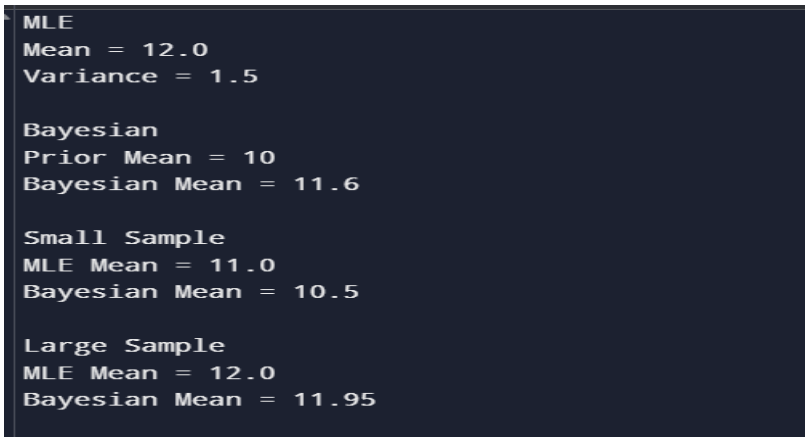
2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

CODE

```
# Dataset

data = [10, 12, 11, 13, 12, 14, 11, 13]

n = len(data)

mean_mle = sum(data) / n

prior_mean = 10

prior_strength = 2

mean_bayes = (

    prior_strength * prior_mean +

    n * mean_mle

) / (prior_strength + n)

print("\nBayesian")

print("Prior Mean =", prior_mean)
```

```
small = [10, 12]
small_mle = sum(small) / len(small)
small_bayes = (
    prior_strength * prior_mean +
    len(small) * small_mle
) / (prior_strength + len(small))
print("\nSmall Sample")
print("MLE Mean =", small_mle)
print("Bayesian Mean =", round(small_bayes, 2))
large = data * 10
large_mle = sum(large) / len(large)
large_bayes = (
    prior_strength * prior_mean +
    len(large) * large_mle
) / (prior_strength + len(large))
print("\nLarge Sample")
print("MLE Mean =", large_mle)
print("Bayesian Mean =", round(large_bayes, 2))
```

OUTPUT



3. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

CODE

```
P_Rain = 0.30
P_S_Rain = 0.10
P_S_NoRain = 0.50
P_W_S = 0.90
P_W_NoS = 0.10
P_Wet = (
    P_Rain * P_S_Rain * P_W_S
    + P_Rain * (1 - P_S_Rain) * P_W_NoS
    + (1 - P_Rain) * P_S_NoRain * P_W_S
    + (1 - P_Rain) * (1 - P_S_NoRain) * P_W_NoS
)
print("P(Wet Grass) =", round(P_Wet, 3))
P_Rain_Wet = (
    P_Rain *
    (
        P_S_Rain * P_W_S
        + (1 - P_S_Rain) * P_W_NoS
P_NotWet = 1 - P_Wet
P_Rain_NotWet = (
    P_Rain *
    (
        P_S_Rain * (1 - P_W_S)
```

```
        + (1 - P_S_Rain) * (1 - P_W_NoS)
print("P(Rain | Not Wet Grass) =",
      round(P_Rain_NotWet, 3))
print("\nAfter changing probability:")
P_W_S = 0.99
P_Wet_New = (
    P_Rain * P_S_Rain * P_W_S
    + P_Rain * (1 - P_S_Rain) * P_W_NoS
    + (1 - P_Rain) * P_S_NoRain * P_W_S
    + (1 - P_Rain) * (1 - P_S_NoRain) * P_W_NoS
)
print("New P(Wet Grass) =",
      round(P_Wet_New, 3))
```

OUTPUT

```
P(Wet Grass) = 0.404
P(Rain | Wet Grass) = 0.134
P(Rain | Not Wet Grass) = 0.413

After changing probability:
New P(Wet Grass) = 0.438
```

4. Implement a concept learning algorithm (such as Candidate Elimination) andVperform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

CODE

```
data = [
    ["Sunny", "Warm", "Normal", "Strong", "Yes"],
```

```
    ["Sunny", "Warm", "High", "Strong", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "No"],
    ["Sunny", "Warm", "High", "Strong", "Yes"]
]
S = ["0", "0", "0", "0"]
G = [["?", "?", "?", "?"]]
for x in data:
    attributes = x[:-1]
    target = x[-1]
    if target == "Yes":
        for i in range(4):
            if S[i] == "0":
                S[i] = attributes[i]
            elif S[i] != attributes[i]:
                S[i] = "?"
    else:
        G = [
            h for h in G
            if any(
                h[i] != "?" and h[i] != attributes[i]
                for i in range(4)
            )
        ]

    print("S =", S)
    print("G =", G)
print("\nHypothesis Space:")
```

```
for n in [2, 3, 4, 5]:
    print("Attributes =", n,
          "Space =", 2 ** n)
```

OUTPUT

```
S = ['Sunny', 'Warm', 'Normal', 'Strong']
G = [['?', '?', '?', '?']]
S = ['Sunny', 'Warm', '?', 'Strong']
G = [['?', '?', '?', '?']]
S = ['Sunny', 'Warm', '?', 'Strong']
G = []
S = ['Sunny', 'Warm', '?', 'Strong']
G = []

Hypothesis Space:
Attributes = 2 Space = 4
Attributes = 3 Space = 8
Attributes = 4 Space = 16
Attributes = 5 Space = 32
```

5. Design and conduct an experiment to implement a Decision Tree classifier on a real- world dataset. Train the model using different splitting criteria such as entropy and Giniindex, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

CODE

```
data = [
    [0, 0, 0],
    [0, 1, 0],
    [1, 0, 1],
    [1, 1, 1],
    [1, 0, 1],
    [0, 0, 0]
```

```
]
def entropy(y):
    p = sum(y) / len(y)
    if p == 0 or p == 1:
        return 0
    return -p*math.log2(p) - (1-p)*math.log2(1-p)
def gini(y):
    p = sum(y) / len(y)
    return 1 - p*p - (1-p)*(1-p)
def predict(x):
    return 1 if x[0] == 1 else 0
actual = [x[2] for x in data]
predicted = [predict(x) for x in data]
accuracy = sum(
    a == p for a, p in zip(actual, predicted)
) / len(actual)
tp = sum(a == p == 1 for a, p in zip(actual, predicted))
fp = sum(a == 0 and p == 1 for a, p in zip(actual, predicted))
fn = sum(a == 1 and p == 0 for a, p in zip(actual, predicted))
precision = tp / (tp + fp) if tp + fp else 0
recall = tp / (tp + fn) if tp + fn else 0
f1 = 2 * precision * recall / (precision + recall)
print("Gini =", round(gini(actual), 2))
print("Entropy =", round(entropy(actual), 2))
print("Accuracy =", round(accuracy * 100, 2), "%")
print("F1 Score =", round(f1, 2))
print("\nTree Depth:")
```

```
for depth in [1, 2, 3]:
    print("Depth", depth, "-> Higher depth may cause overfitting")
print("\nPruning: Reduces tree size and overfitting")
```

OUTPUT

```
Gini = 0.5
Entropy = 1.0
Accuracy = 100.0 %
F1 Score = 1.0

Tree Depth:
Depth 1 -> Higher depth may cause overfitting
Depth 2 -> Higher depth may cause overfitting
Depth 3 -> Higher depth may cause overfitting

Pruning: Reduces tree size and overfitting

=== Code Execution Successful ===
```